

07 — CORE LANGUAGE ESSENTIALS

The foundations interviewers assume you have airtight: const-correctness, initialization, linkage/ODR, and exceptions.

How to think about this. These topics feel like a grab-bag but share one theme: *what the compiler guarantees vs. what you must guarantee yourself*. `const` is a promise you make that the compiler enforces. Initialization rules decide whether a value is defined or garbage. Linkage/ODR govern what "the same entity" means across files. Exception safety is a contract about state after a failure. Learn each as "who promises what to whom," and the trivia (most vexing parse, `inline` semantics, `noexcept`) stops being trivia.

Take-home lesson. The "boring" core — `const`, initialization, linkage, exception safety — is where senior candidates separate from mid-level ones. Anyone can describe a `vector`; the SDE-2 signal is knowing *why* `Widget w()` declares a function and *what* `noexcept` promises the optimizer.

1. `CONST` / `CONSTEXPR` CORRECTNESS

- `const` on a member function = "doesn't modify observable state"; enables calling on const objects. `mutable` members may still change (caches, mutexes).
- East vs west const cosmetic; `const int` == `int const`. But pointer const is positional:
 - `const int* p` — pointer to const int (can't change `*p`).
 - `int* const p` — const pointer to int (can't change `p`).
 - `const int* const p` — both.
- Top-level vs low-level const matters for overload resolution & deduction.
- `constexpr` implies `const` for variables; guarantees compile-time value.

- const-correctness propagates: design APIs const-first; it's cheap to add early, painful to retrofit.
-

2. REFERENCES VS POINTERS

	Reference	Pointer
Rebind	no (bound once)	yes
Null	no (must bind)	yes
Arithmetic	no	yes
Syntax	value-like	explicit deref

- Use a reference when it must always refer to a valid object and never rebind; a pointer (or `optional`) when it can be null/rebindable.
 - `const T&` params avoid copies for read-only; but for cheap types (int, small structs) pass by value. For sink params take by value + move.
-

3. INITIALIZATION (A GENUINE MINEFIELD)

- Forms: copy `T x = v;`, direct `T x(v);`, brace `T x{v};`, aggregate, value `T x{};`, default `T x;`.
- Prefer brace `{}`: prevents narrowing conversions (`int x{3.5}` is an error), and value-initializes. But beware the `std::vector<int> v{3, 1}` vs `v(3, 1)` trap: braces prefer the `initializer_list` ctor (2 elements), parens call (3 copies of 1).
- Most vexing parse: `Widget w();` declares a *function*, not an object. `Widget w{};` avoids it.
- Uninitialized locals of built-in type have indeterminate value (reading = UB). Prefer always initializing.
- Member init order = declaration order; use the member initializer list (not assignment in the ctor body) for const/reference members and to avoid

double-init.

Stop and think. What does this print, and why?

```
struct S {  
    int a;  
    int b;  
    S(int x) : b(x), a(b) {}    // note: b listed first in the init list  
};  
S s{5}; // s.a == ? s.b == ?
```

► answer

4. ODR, LINKAGE & THE INLINE STORY

- One Definition Rule: exactly one definition of each entity across the program (some may be repeated identically in headers if `inline`).
- Internal linkage: `static` at namespace scope, or anonymous namespace → visible only in that TU. External linkage: default for non-const globals, functions.
- `inline` today means "may be defined in multiple TUs identically", i.e. it relaxes ODR — *not primarily* a request to inline code. Header-defined functions and (C++17) `inline` variables rely on this.

`inline` is an ODR keyword, not a speed hint. It permits one definition to appear identically in multiple translation units and tells the linker to merge them into a single entity. Whether a call is *actually* inlined is a separate optimizer decision made regardless of the keyword. This is why header-only libraries mark functions `inline`, and why C++17 `inline` variables are coherent — you can't "inline a variable for speed"; the word is purely about linkage.

- `extern "C"` — C linkage (no name mangling) for interop.
- Prefer anonymous namespaces over file-scope `static` for internal linkage in modern code.

5. EXCEPTIONS & ERROR HANDLING

- Stack unwinding: when an exception propagates, destructors of fully-constructed automatic objects run in reverse order → RAII cleans up.
- Exception safety guarantees (state these precisely):
 - *No-throw* (`noexcept`) — never fails.
 - *Strong* — commit-or-rollback; state unchanged on failure (copy-and-swap).
 - *Basic* — invariants preserved, no leaks, but state may change.
 - *None* — avoid.
- `noexcept` — a promise; violating it calls `std::terminate`. Enables optimizations (esp. move ops in containers). Destructors are implicitly `noexcept` — throwing from a destructor during unwinding = terminate.
- Never let an exception escape a destructor.
- `throw;` rethrows the current exception (preserves type). Catch by `const&` to avoid slicing the exception object.
- Error-code alternatives: `std::optional` (maybe-a-value), `std::expected<T,E>` (C++23, value-or-error), `std::error_code`. Choose exceptions for exceptional/unrecoverable, value-based for expected failures in hot paths.

```
// C++23 std::expected — error handling without exceptions, composable
via monads:
std::expected<int, std::string> parse(std::string_view s) {
    if (s.empty()) return std::unexpected("empty input");
    return std::stoi(std::string{s});
}
auto doubled = parse("21")
    .transform([](int x){ return x * 2; }) // runs only on
success
    .or_else([](std::string e){           // runs only on error
        return std::expected<int, std::string>{0};
    });
```

Take-home lesson. Exceptions vs `expected` is a *frequency* decision: exceptions are (near) zero-cost on the success path but expensive when thrown — perfect for genuinely rare failures. `expected` / `optional` make failure an ordinary return value — perfect for expected, hot-path failures (parsing, lookups) where you don't want the throw cost or the hidden control flow.

6. `ENUM CLASS` , `AUTO` , MISC MODERN HYGIENE

- `enum class` (scoped) — strongly typed, no implicit int conversion, no scope pollution. Prefer over plain `enum`. Specify underlying type when it matters for ABI/size: `enum class E : uint8_t`.
 - `auto` — deduces (drops top-level const/ref like by-value template deduction); `auto&` , `const auto&` , `auto&&` , `decltype(auto)` for exact. Use for obvious/verbose types and iterators; keep it readable.
 - `nullptr` not `NULL` / `0` (type-safe, no overload ambiguity).
 - Uniform use of `using` over `typedef`; `using` supports templates (alias templates).
 - Structured bindings (C++17): `auto [a, b] = pair;`
 - `[[nodiscard]]` , `[[maybe_unused]]` , `[[fallthrough]]` attributes.
-

7. LAMBDA (KNOW THE DESUGARING)

- A lambda is an unnamed closure class with `operator()`; captures become members. `[=]` copy, `[&]` reference, `[x]` / `[&x]` explicit, `[this]` / `[*this]` (C++17 copy of `*this`), `[x = expr]` init-capture (C++14, enables *move* into a lambda).
- Dangling capture is the top lambda bug: `[&]` capturing a local that outlives the lambda (e.g. stored in a container / returned). Capture by copy or extend lifetime.

```
// init-capture (C++14) to MOVE a move-only object into a lambda:
auto task = [p = std::move(uptr)]() mutable { p->run(); }; // owns the
unique_ptr
// [&] here would dangle once the enclosing scope returns; [=] can't copy
uptr.

// C++20: capture *this by value to avoid a dangling this-pointer in
async work:
auto job = [*this]{ return compute(); }; // copies the whole object,
safe to outlive
```

- `mutable` allows modifying by-copy captures. Generic lambdas: `[] (auto x) {}`.
 - `constexpr` lambdas (C++17). A capture-less lambda converts to a function pointer.
-

SELF-CHECK LADDER (GRADE YOUR OWN MASTERY)

Level 1 — Recall

- What's the difference between top-level and low-level `const`?
- Name the four exception-safety guarantees.
- What does `inline` actually control (in C++17 terms)?

Level 2 — Apply

1. Read aloud and explain `const int* const`, `int const *`, `char* const*`.
2. Give the most vexing parse and two fixes.
3. Show a dangling-capture lambda bug and fix it with init-capture.

Level 3 — Transfer

4. Why does `std::vector<int> v{3,1}` differ from `v(3,1)`? Explain via `initializer_list` overload preference.
5. Make a function strong-exception-safe via copy-and-swap; explain *why* it's strong.
6. Why does throwing from a destructor terminate the program during unwinding? Reason

from "two exceptions in flight."

Level 4 — Teach

- Explain const-correctness to someone who thinks it's just "extra typing." If you can't show a bug it prevents *and* an optimization it enables, keep going.

CONNECTS TO

- File 01 — RAI and stack unwinding are the mechanism behind exception safety.
- File 02 — `noexcept` on moves; the exception guarantees that drive move-vs-copy in containers.
- File 05 — RAI lock guards, `noexcept` on thread entry points.
- File 08 — `optional` / `expected`, structured bindings, `inline` variables, `enum class` improvements are the modern-standard evolution of this core.